

DRIP: Defending Prompt Injection via Token-wise Representation Editing and Residual Fusion

A Artifact Appendix

A.1 Abstract

This artifact provides the full implementation of DRIP, a training-time defense against prompt injection for decoder-only LLMs. It includes:

- (1) DPO fine-tuning scripts for LLaMA-3-8B-Instruct, Llama-3.1-8B-Instruct, and Mistral-7B-Instruct-v0.3 with the de-instruction shift and residual re-instruction fusion pathway;
- (2) Evaluation scripts for robustness benchmarks (SEP, AlpacaFarm injection, InjecAgent, GCG, PAIR, TAP, PISmith, AgentDojo) and utility benchmarks (AlpacaEval-2.0, IFEval, MT-Bench, MMLU);
- (3) Curated training data archived on Zenodo (DOI: <https://doi.org/10.5281/zenodo.20603331>); and
- (4) Pretrained LoRA adapter checkpoints released on HuggingFace.

The artifact is licensed under MIT (see LICENSE in the repository).

A.2 Description & Requirements

The repository is organized as follows:

- `README.md`: End-to-end setup and evaluation guide.
- `setup_env.sh`: Conda environment setup script.
- `config.py`: Chat template configuration.
- `scripts/`: Shell scripts for training and evaluation, organized by model (llama8b/, mistral7b/) and task (sep/, alpaca/, agentdojo/, evaluation/).
- `training/`: DPO training pipeline and LoRA merge utility.
- `testing/`: Evaluation modules for SEP, InjecAgent, MT-Bench, IFEval, MMLU, AgentDojo, PAIR, TAP, and PISmith.
- `gcg/`: GCG adaptive attack evaluation (requires a separate legacy environment).
- `data_generation/`: Scripts to regenerate DPO training data from scratch.
- `datasets/`: Curated training and evaluation data (downloaded from Zenodo), including a manifest (MANIFEST.md) listing every file with its SHA-256 checksum.

A.2.1 Security, privacy, and ethical concerns. The artifact includes scripts that simulate prompt injection attacks (GCG, PAIR, TAP, PISmith, AgentDojo) against LLMs. These are strictly for research evaluation. No real user data is used; all datasets are publicly available benchmarks. Evaluators should not run attack scripts against production systems.

A.2.2 How to access.

- Code: <https://github.com/lindsey98/PromptInjection>, archived on Zenodo (DOI: <https://doi.org/10.5281/zenodo.20695257>)
- Training data: Zenodo (DOI: <https://doi.org/10.5281/zenodo.20603331>)
- Pretrained LoRA adapters on HuggingFace (Kelsey98/), pinned to the exact revisions evaluated in the paper:

- Meta-Llama-3-8B-Instruct-TextTextText-drip (3-role chat template, for text-to-text attacks only; revision <COMMIT-HASH-1>)
- Mistral-7B-Instruct-v0.3-TextTextTextMistral-drip (3-role chat template, for text-to-text attacks only; revision <COMMIT-HASH-2>)
- Llama-3.1-8B-Instruct-TextTextText-4roles-toolcall-drip (4-role chat template, for both text-to-text attacks and tool-call attacks; revision <COMMIT-HASH-3>)

SHA-256 checksums of the adapter weights are listed in `datasets/MANIFEST.md`.

Text-to-text vs. tool-call attacks. In the text-to-text setting, both the primary task and the injected task are text-based (e.g., translation or summarization): the model is expected to produce a natural-language output, and the attacker’s goal is to divert the model into producing a different text response than intended. In the tool-call setting, the model operates as an agent that can either emit a tool invocation (specifying a function and its arguments to execute in the next step) or produce a final text answer. The attacker’s goal is to inject a malicious directive into the tool output so that the model calls a harmful function instead of completing the legitimate task. Because the output space differs fundamentally between the two settings, we train and evaluate separate checkpoints: the 3-role checkpoints cover text-to-text attacks, while the 4-role checkpoint additionally handles tool-call attacks evaluated on InjecAgent and AgentDojo.

A.2.3 Hardware dependencies. Training uses 8× NVIDIA RTX 5880 Ada (49 GB each) with `torch==2.8.0` and CUDA runtime 12.8; at minimum, 6 GPUs with ≥48 GB VRAM each are required. The GPU count is configurable in the training launchers. Evaluation runs on a single GPU with ≥24 GB VRAM; ≥48 GB is recommended for the adaptive attack benchmarks (GCG, PAIR, TAP, PISmith). An OpenAI API key is required for SEP scoring, AlpacaEval-2.0, and MT-Bench (AlpacaEval-2.0 may cost up to USD 50).

A.2.4 Software dependencies.

- `conda`: run `bash setup_env.sh` to install all pinned dependencies.
- `torch==2.8.0`: install manually following the PyTorch official guide if CUDA version differs.
- GCG evaluation requires a separate legacy transformers environment (see `gcg/README.md`).
- OpenAI API key configured via `datasets/openai_configs.yaml`.

A.2.5 Benchmarks.

- **SEP**: Structured evaluation of role separation; higher score is better.
- **AlpacaFarm injection**: ASR under Naive, Ignore, Completion, Escape, and HackaPrompt attacks.

- **InjecAgent**: Agentic injection benchmark; ASR under direct and enhanced attacks.
- **GCG / PAIR / TAP / PISmith**: Adaptive adversarial attacks; ASR.
- **AgentDojo**: End-to-end agentic task benchmark; utility and ASR under 4-role setting.
- **AlpacaEval-2.0**: Win rate against GPT-4 on instruction following.
- **IFEval**: Strict instruction-following accuracy.
- **MT-Bench**: Multi-turn quality scored by GPT-4.
- **MMLU**: Knowledge benchmark.

A.3 Setup

A.3.1 Installation. Install the requirements, base model, datasets, and setup the OpenAI key.

```
git clone https://github.com/lindsey98/PromptInjection
cd PromptInjection
# Download base model
huggingface-cli download meta-llama/Llama-3.1-8B-Instruct \
  --local-dir meta-llama/Llama-3.1-8B-Instruct \
  --resume-download --local-dir-use-symlinks False
# Create conda environment
bash setup_env.sh
conda activate prompt
# Download curated training/evaluation data
wget -O datasets.zip \
  "https://zenodo.org/records/20603331/files/datasets.zip?download=1"
unzip datasets.zip && mv datasets1/ datasets/
# Configure OpenAI key
cp datasets/openai_configs_example.yaml datasets/
openai_configs.yaml
```

A.3.2 Basic test. Download a pretrained adapter at its pinned revision, merge it into the base model, and verify it loads correctly:

```
REPO=Llama-3.1-8B-Instruct-TextTextText-4roles-toolcall-drip
huggingface-cli download Kelsey98/$REPO \
  --revision <COMMIT-HASH-3> --local-dir $REPO
CUDA_VISIBLE_DEVICES=0 python -m training.merge_lora \
  --adapter_path "$REPO/" \
  --output_path "$REPO-merged/" \
  --base_model_path meta-llama/Llama-3.1-8B-Instruct \
  --customized_model_class LlamaForCausalLM
```

Expected: the merged model loads without errors and responds to a prompt by following the system instruction rather than an injected command in the tool output turn.

A.4 Evaluation Workflow

A.4.1 Major claims.

- (C1): DRIP outperforms state-of-the-art defenses (StruQ, SecAlign, ISE, PFT) on SEP score, AlpacaFarm ASR, and InjecAgent ASR, improving role separation by 12–49% and reducing ASR by over 66%, while preserving utility on AlpacaEval-2.0, IFEval, MT-Bench, and MMLU. This is shown in Tables 1–3 of the paper.

- (C2): DRIP maintains robustness against adaptive attacks (GCG, PAIR, TAP, PISmith) and in agentic settings, reported in Table 4.

A.4.2 Experiments.

- (E1): **Robustness: SEP + AlpacaFarm + InjecAgent** [30 person-minutes + ≥ 38 compute-hours on 1×GPU]: Evaluates (C1). **Preparation:** Download and merge the 3-role adapter Kelsey98/Meta-Llama-3-8B-Instruct-TextTextText-drip (base model meta-llama/Meta-Llama-3-8B-Instruct), following the same procedure as the basic test in the Setup section.

Execution: Run the evaluation.

```
# SEP
bash scripts/evaluation/llama8b/sep.sh
python testing/sep/sep_judge.py
python testing/sep/sep_collect.py
# AlpacaFarm injection
bash scripts/evaluation/llama8b/alpaca_injection.sh
python testing/evaluation_main.py -m [model_path]
# InjecAgent
bash scripts/evaluation/llama8b/injecagent.sh
```

Results: SEP score, AlpacaFarm ASR (5 attack types), and InjecAgent ASR should match Tables 1–2 within $\pm 1\%$.

- (E2): **Utility: AlpacaEval-2.0 + IFEval + MT-Bench + MMLU** [30 person-minutes + 5 compute-hours + OpenAI API cost]: Evaluates (C1).

Preparation: Ensure datasets/openai_configs.yaml is populated.

Execution: Run the evaluation.

```
bash scripts/evaluation/llama8b/alpaca_utility.sh
bash scripts/evaluation/llama8b/ifeval.sh
bash scripts/evaluation/llama8b/mtbench.sh
bash scripts/evaluation/llama8b/mmlu_utility.sh
```

Results: Utility scores should be comparable to the base model (within $\pm 2\%$), reproducing Table 3.

- (E3): **Adaptive attacks: GCG + PAIR/TAP/PISmith + AgentDojo** [1 person-hour + ≥ 84 compute-hours on 2×GPU]: Evaluates (C2).

Preparation: GCG requires a separate legacy environment (see gcg/README.md). AgentDojo uses the merged 4-role checkpoint (repo: Kelsey98/Llama-3.1-8B-Instruct-TextTextText-4roles-toolcall-drip).

Execution: Follow gcg/README.md, testing/pair/README.md, testing/tap/README.md, testing/pismith/README.md, and testing/agentdojo/README.md respectively.

Results: ASR under all adaptive attacks should be substantially lower than the undefended baseline, reproducing Table 4.

A.5 Notes on Reusability

DRIP consists of two independent components: the de-instruction shift (DPO fine-tuning) and the residual re-instruction fusion. Both can be applied to any decoder-only transformer supported by the transformers library. The 3-role and 4-role checkpoints cover text instruction-following and tool-calling agents respectively. For new

base models, adapt the delimiter configuration in `config.py` and rerun data curation via `data_generation/README.md`. Note that the de-instruction shift layer is designed for dense architectures; for MoE backbones, the residual fusion and data-curation recipe can be used independently without the representation editing layer.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.